

Article

SST-YOLO: An Improved Autonomous Driving Object Detection Algorithm Based on YOLOv8

Qinsheng Du ^{1,2,3,*}, Ningbo Zhang ^{1,3}, Wenqing Bi ^{1,3}, Ruidi Zhu ^{1,3}, Yuhan Liu ^{1,3}, Chao Shen ^{1,2}, Shiyan Zhang ^{1,3} and Jian Zhao ^{1,2,3,*} 

¹ College of Computer Science and Technology, Changchun University, Satellite Road, Changchun 130022, China; duqs@ccu.edu.cn (Q.D.); znb1999@sina.com (N.Z.); biwenqing0506@gmail.com (W.B.); zrddyx12138@126.com (R.Z.); 18342137879@163.com (Y.L.); shenchao3840626@163.com (C.S.); myes0622@163.com (S.Z.)

² Jilin Rehabilitation Equipment and Technology Engineering Research Center for the Disabled, Changchun University, Satellite Road, Changchun 130022, China

³ Ministry of Education Key Laboratory of Intelligent Rehabilitation and Barrier-Free Access for the Disabled, Changchun University, Satellite Road, Changchun 130022, China

* Correspondence: zhaojian@ccu.edu.cn

Abstract

As autonomous driving technology progresses, efficient and accurate object detectors are able to detect pedestrians, vehicles, road signs, and obstacles in real time, thereby enhancing driving safety and serving as a part of autonomous driving. However, the performance of such object detectors is limited and cannot be leveraged to satisfy modern autonomous driving systems. To address this issue, we develop an object detection network for autonomous driving scenarios, SST-YOLO, which is based on YOLOv8. First, we propose a Sobel Convolution & Convolution (SCC) module to enhance the backbone, which incorporates a SobelConv branch to explicitly model gradient-based edge information and improve structural feature representation. In addition, we replace the original path aggregation feature pyramid network (PAFPN) with a Small Object Augmentation Pyramid Network (SOAPN), which integrates SPDCConv and CSP-OmniKernel modules to strengthen multi-scale feature fusion and enhance small object representation. Finally, a Task-Adaptive Decomposition & Alignment Head (TADAHead) is designed, which employs task decomposition, dynamic deformable convolution, and classification-aware modulation to decouple tasks and achieve adaptive spatial alignment, thereby improving detection accuracy and robustness in complex scenarios. Experiments on the public autonomous driving dataset KITTI show that our proposed method outperforms the baseline YOLOv8 model. Compared with the baseline results, mAP@0.5:0.95 ranges from 65.1% to 69.2%, which indicates that the proposed SST-YOLO network can achieve object detection for autonomous cars.

Keywords: YOLOv8; autonomous driving; Sobel convolution; SOAPN; TADAHead



Academic Editors: Deokwoo Lee and Yohan Park

Received: 20 February 2026

Revised: 26 March 2026

Accepted: 31 March 2026

Published: 2 April 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Autonomous driving [1] has become a representative application of computer vision [2], where object detection [3] serves as a core component for perceiving surrounding environments. It aims to accurately localize and classify traffic participants, such as vehicles, pedestrians, and traffic signs, from images or point clouds [4]. In safety-critical scenarios, detection performance directly impacts downstream decision-making, requiring both high accuracy and real-time capability under complex conditions, including scale variation, occlusion, and illumination changes.

Despite recent progress, object detection in real-world driving environments remains challenging. Dense scenes, adverse weather, and long-range perception significantly degrade detection reliability. In particular, small objects [5] and distant targets [6] are prone to feature loss and insufficient representation, leading to missed detections. These limitations highlight the need for more effective feature extraction and multi-scale representation mechanisms.

Deep learning-based methods [7], especially convolutional neural networks (CNNs) [8], have substantially improved detection performance by enabling hierarchical feature learning. Existing detectors are typically divided into two-stage methods [9] and single-stage methods [10]. While two-stage detectors achieve high accuracy through region proposal and refinement, their computational complexity limits real-time applicability. In contrast, single-stage detectors (e.g., SSD and YOLO) offer faster inference and end-to-end optimization [11], making them more suitable for autonomous driving. However, their performance on small and distant objects remains suboptimal, particularly in complex environments.

To address these issues, this paper proposes SST-YOLO, an improved detector based on YOLOv8, designed to enhance feature representation and task-specific optimization. Specifically, a Sobel Convolution & Convolution (SCC) module is introduced to incorporate gradient-based edge information [12] into the backbone, strengthening structural feature extraction. In addition, a Small Object Augmentation Pyramid Network (SOAPN) is designed to improve multi-scale feature fusion, and a Task-Adaptive Decomposition & Alignment Head (TADAHead) is developed to enhance regression and classification consistency.

The main contributions of this work are summarized as follows:

- (1) An SCC module is proposed to integrate Sobel-based gradient features with conventional convolution, improving edge awareness and feature diversity.
- (2) A SOAPN structure is designed to enhance multi-scale representation, particularly for small object detection.
- (3) A TADAHead is introduced to achieve task-adaptive feature decoupling and spatial alignment, improving detection accuracy.
- (4) Experimental results on the KITTI dataset demonstrate consistent improvements over the YOLOv8 baseline.

2. Related Work

2.1. Overview of Object Detection

Object detection aims to identify object categories and localize their positions in images, and it is a fundamental task in computer vision with applications in autonomous driving, surveillance, and medical imaging. Due to variations in object appearance, scale, pose, illumination, and occlusion, it remains a challenging problem. Existing methods are broadly categorized into two-stage and single-stage detectors. Two-stage methods generate candidate regions followed by classification and regression, whereas single-stage methods directly predict object locations and categories from input images.

2.2. Two-Stage Object Detection Algorithms

R-CNN [13] generates region proposals via selective search (SS) [14], followed by CNN-based feature extraction and classification, with nonmaximum suppression (NMS) [15] applied to obtain final detections. However, redundant proposals lead to a high computational cost. Fast R-CNN [16] integrates the advantages of R-CNN and SPPNet [17], improves efficiency by sharing convolutional computation and introducing RoI pooling, while replacing SVM [18] with Softmax for end-to-end training. Faster R-CNN [19] further introduces a region proposal network (RPN), which generates proposals through anchor-

based classification, significantly improving speed and proposal quality. Despite their accuracy, two-stage methods remain computationally intensive.

2.3. Single-Stage Object Detection Algorithms

YOLO [20] formulates detection as a regression problem by dividing images into grids, enabling fast inference but suffering from limited generalization and poor small-object performance. SSD [21] introduces multiscale feature maps for detection at different resolutions, improving performance on objects of varying sizes. However, it increases computational complexity and lacks sufficient semantic representation in shallow layers. Overall, single-stage detectors achieve better speed–accuracy trade-offs but still struggle with small and distant objects.

2.4. YOLO Network

The YOLO series has evolved significantly. YOLOv1 introduced end-to-end detection, while YOLOv2 and YOLOv3 improved accuracy through deeper architectures and residual connections. YOLOv4 incorporated CSPNet to reduce redundancy and PAN for multi-scale fusion. YOLOv5 and YOLOv6 improved efficiency and deployment, while YOLOv7 further enhanced feature fusion and performance. YOLOv8 adopts a simplified architecture with improved feature fusion and loss design. Subsequent versions (YOLOv9–YOLOv11) integrate advanced techniques such as attention mechanisms and Transformer-based modules. It is essential to balance detection speed and accuracy when selecting the most appropriate object detection framework for a given research scenario. Considering the stability, widespread adoption, and clear modular design of YOLOv8, it is selected as the baseline model for SST-YOLO. YOLOv8 provides a well-balanced and structurally transparent framework (whether compared with earlier or newer versions), which facilitates fair evaluation of the proposed improvements in autonomous driving scenarios.

2.5. YOLOv8

The network architecture of YOLOv8 consists of the following three main components:

Backbone: The backbone is responsible for feature extraction and is composed of a series of convolutional and deconvolutional layers. Residual connections and bottleneck structures are employed to reduce network complexity while enhancing performance.

Neck: The neck utilizes multiscale feature fusion to integrate feature maps from different stages of the backbone, thereby enhancing feature representation.

Head: The head is responsible for the final object detection and classification tasks. It consists of a detection head and a classification head. The detection head comprises a series of convolutional and deconvolutional layers to generate detection results, whereas the classification head employs global average pooling to perform category prediction for each feature map.

The overall architecture of YOLOv8 is illustrated in Figure 1. The backbone is mainly composed of multiple convolutional layers and C2f modules, which progressively extract hierarchical features from the input image. The neck integrates features from different stages through modules such as SPPF and PAN-based structures, enabling effective multiscale feature fusion. Finally, the head decouples classification and regression tasks and outputs object categories and bounding box coordinates. Although YOLOv8 achieves strong efficiency, its performance on small objects remains limited, motivating further improvements.

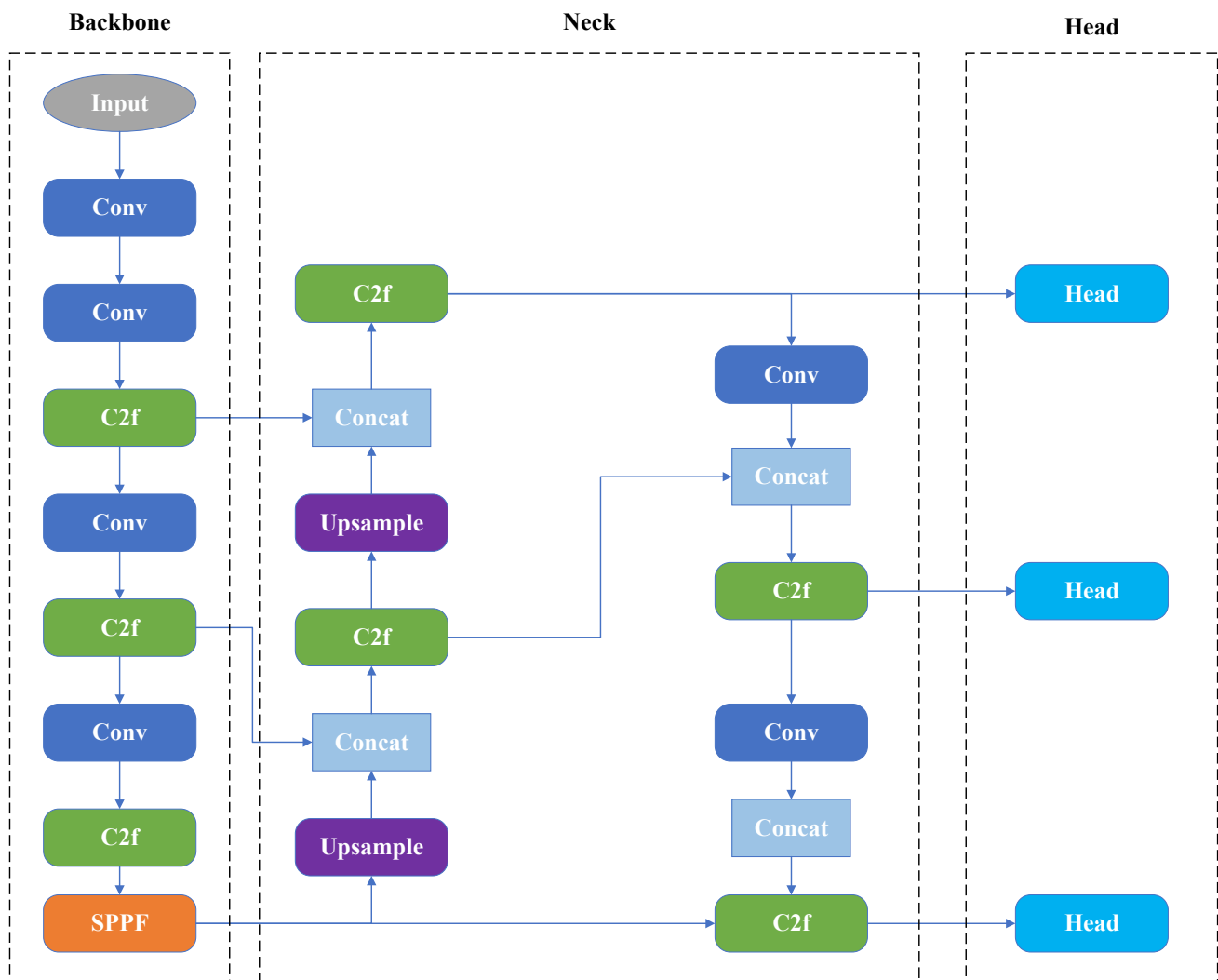


Figure 1. Structural diagram of YOLOv8.

3. Method

To address the limitations of YOLOv8 in complex driving scenarios, particularly for small and distant object detection, we propose SST-YOLO, an enhanced detection framework that improves feature extraction, multiscale fusion, and prediction. Its structural diagram is shown in Figure 2. First, the backbone incorporates the SCC module into C2f-based structures to extract hierarchical features enriched with edge and structural information. The resulting features are then transmitted to the neck, where the SOAPN, built upon SPDCConv and CSP-OmniKernel, performs multiscale fusion by effectively combining shallow and deep representations. Finally, the fused features are fed into the TADAHead, which generates detection outputs through task-adaptive decoupling and spatial alignment of classification and regression. Through this progressive process from feature extraction to prediction, SST-YOLO forms a unified and optimized detection framework.

3.1. SCC

In autonomous driving object detection tasks, the effectiveness of feature extraction is critical to both real-time performance and detection accuracy. Although the C2f (cross-stage partial fusion) module can effectively facilitate feature fusion, it still has several limitations. First, C2f essentially remains a conventional convolutional stacking structure and lacks explicit mechanisms for extracting image gradients and edge features. However, the most critical targets in autonomous driving scenarios—such as pedestrian contours, vehicle

boundaries, lane markings, and traffic sign edges—often rely on clear edge information for accurate recognition. This limitation leads to insufficient sensitivity of C2f to small objects and weak-texture targets.

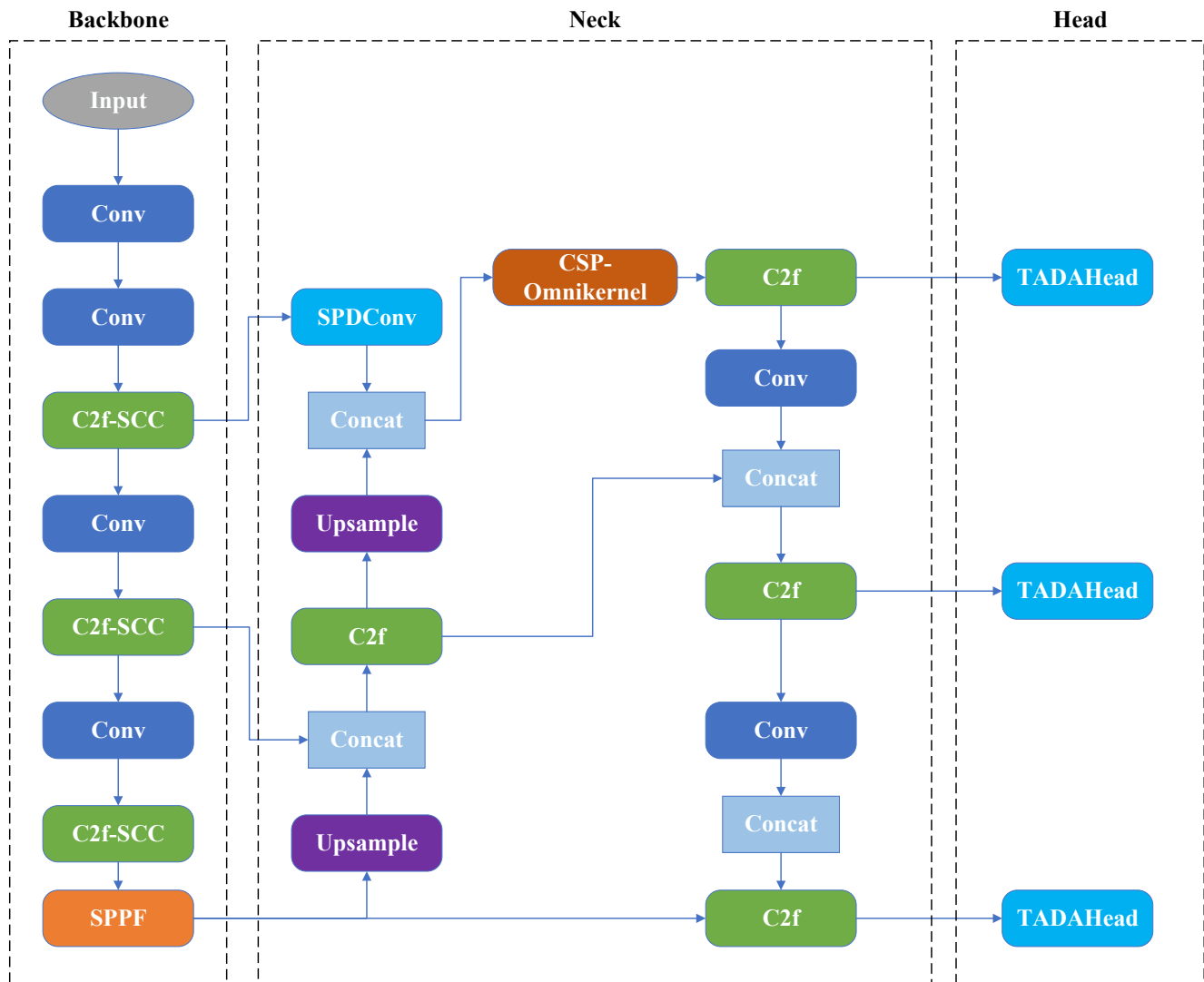


Figure 2. Structural diagram of SST-YOLO.

Second, C2f relies solely on standard convolutions for texture feature extraction without introducing additional structural enhancements. In real-world road environments, where occlusions, rain or fog, and low image clarity are common, local geometric structures become more crucial. Moreover, closely adjacent objects require stronger edge discrimination capabilities. The lack of structural diversity in the traditional C2f module and its insensitivity to fine-grained texture details make it difficult to meet these demands.

Finally, C2f is not well suited for small-scale object detection. In autonomous driving scenarios, many key targets—such as pedestrians, cyclists, traffic lights, and road signs—often occupy only tens of pixels. After downsampling, the convolutional operations in C2f tend to lose critical edge features, resulting in high miss detection rates for small objects and large localization errors for distant targets.

To address these issues, this study introduces an SCC (SobelConv–Conv) module constructed via the Sobel operator, which is used to improve the traditional C2f module, forming a new C2f-SCC module. The structures of the SCC module and the C2f-SCC module are illustrated on the left and right sides of Figure 3, respectively. The proposed module is capable of extracting features from the original image while preserving rich

spatial information, and it can effectively capture abrupt intensity changes to obtain critical edge information.

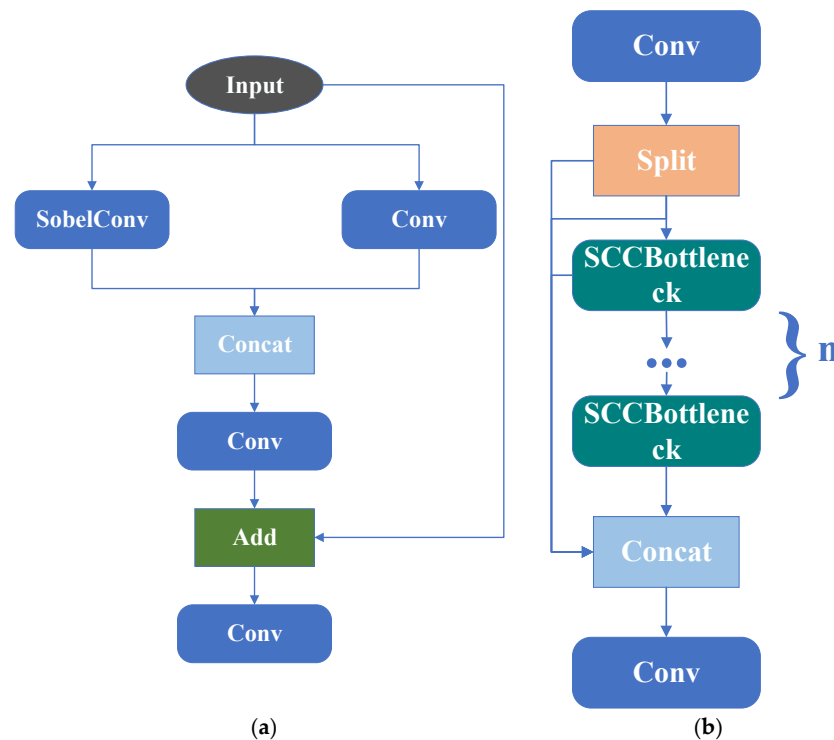


Figure 3. Structural diagram. (a) SCC; (b) C2f-SCC.

The SCC module introduces a SobelConv branch to extract edge features explicitly. The Sobel operator is a classical edge detection operator, it achieves edge detection by calculating the local gray-level gradient of an image in the horizontal and vertical directions, highlighting edges and intensity changes, thereby highlighting important edge information. From the perspective of feature frequency characteristics, the Sobel operator primarily responds to high-frequency components in an image, which correspond to edges and intensity discontinuities. By computing local gradients in the horizontal and vertical directions, it explicitly emphasizes structural variations and boundary information that are closely related to object contours. In contrast, the convolutional filters learned in modern deep neural networks are optimized through data-driven training and tend to capture more complex semantic patterns and contextual dependencies across different receptive fields. As a result, gradient-based responses and learned convolutional features characterize image content from different representational perspectives. Introducing a Sobel-based branch enhances the representation of structural details while allowing convolutional layers to focus on higher-level feature abstraction, leading to more informative and discriminative feature representations for subsequent detection tasks. Its detailed principle is illustrated in Figure 4 (where Sobel-x denotes the gradient in the horizontal direction, and Sobel-y denotes the gradient in the vertical direction).

In addition to edge features, spatial information is equally important. Therefore, SCC incorporates an additional convolutional branch to extract features from the original image, preserving rich spatial details.

Finally, features extracted from the SobelConv and standard convolution branches are concatenated, enabling the learned representation to encode rich edge information and spatial features simultaneously, thus providing a more comprehensive characterization of the image content.

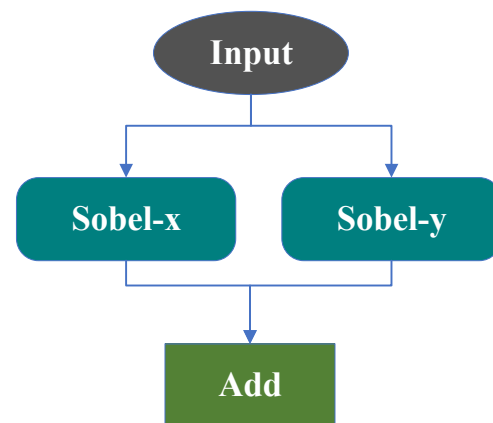


Figure 4. Schematic diagram of SobelConv.

The workflow of the proposed module is described below.

Given an input feature map X extracted from the original image, the feature map is first fed into the SobelConv branch, where the Sobel operator is applied to compute edge information in the horizontal and vertical directions. Let S_x and S_y denote the horizontal and vertical Sobel kernels, respectively. The edge feature X_{sobel} is computed as:

$$X_{\text{sobel}} = |X * S_x| + |X * S_y|,$$

where $*$ denotes the convolution operation.

Moreover, the input feature map X is processed by the Conv branch, which extracts features from the original image via standard 3×3 convolution. The resulting feature is denoted as:

$$X_{\text{conv}} = f_{\text{conv}}(X),$$

where f_{conv} represents the 3×3 convolution operation.

The outputs of the SobelConv branch and the Conv branch are subsequently concatenated along the channel dimension to obtain the fused feature X_{concat} :

$$X_{\text{concat}} = [X_{\text{sobel}} \cdot X_{\text{conv}}],$$

where $[x_1 \cdot x_2]$ denotes channelwise concatenation.

To integrate the fused information and reduce channel redundancy, a 1×1 convolution is applied for channel compression, yielding the integrated feature X_{feature} :

$$X_{\text{feature}} = f_{1 \times 1}(X_{\text{concat}}),$$

where $f_{1 \times 1}$ denotes the 1×1 convolution operation.

Finally, another 1×1 convolution is applied after introducing a residual connection with the original input feature map X , resulting in the enhanced feature map X' :

$$X' = f_{1 \times 1}(X_{\text{feature}} + X),$$

where the residual connection ensures information completeness and stabilizes feature learning.

By incorporating the Sobel operator, the proposed module explicitly enhances the model's ability to capture edge information. Compared with the original C2f module in YOLOv8, the improved module achieves higher detection accuracy and recall, thereby providing a solid foundation for object detection tasks in autonomous driving scenarios.

3.2. SOAPN

A multiscale feature fusion network, the path aggregation feature pyramid network (PAFPN) [22], which is widely adopted in the YOLO series, enables bidirectional fea-

ture interaction through top-down and bottom-up pathways. However, for diverse road object types in autonomous driving environments—such as pedestrians, cyclists, long-distance traffic signs, low-reflective obstacles at night, and nonrigid objects occluded by vehicles—the traditional PAFPN architecture exhibits several inherent structural bottlenecks, making it insufficient for meeting the robustness requirements of complex road scenes.

First, PAFPN relies heavily on a full-channel bidirectional propagation mechanism during feature fusion. High-level semantic features are progressively upsampled and transmitted to lower layers in a linear topological manner, which leads to gradual dilution of deep semantic information during reverse propagation, making it difficult to adequately recover fine-grained texture details. Second, the convolutional kernel sizes in PANet are fixed. When facing traffic targets with extreme scale variations (e.g., pedestrians at a distance of 50 m versus traffic cones within 1 m), the limited receptive field makes it difficult to balance global contour perception and local texture extraction, resulting in fragmented and blurred feature representations. Third, PAFPN emphasizes single-path feature fusion and lacks cross-scale redundant information feedback mechanisms. As a result, in dynamic and complex scenarios—such as curved lane markings, vehicle interactions, and illumination degradation under rainy or foggy conditions—the network exhibits insufficient robustness to occlusion and reflective noise.

On the basis of the above analysis, this study redesigns the feature fusion pathway of YOLOv8 and proposes the small object-oriented augmentation pyramid network (SOAPN) as a replacement for PAFPN. SOAPN enhances the bidirectional representation of global semantics and local details through heterogeneous-scale convolutions, multidimensional receptive field mixing, and lightweight cross-layer fusion. Consequently, the detection network demonstrates stronger sensitivity to small objects, improved long-distance recognition capability, and enhanced robustness to occlusion in autonomous driving scenarios.

The SOAPN architecture consists of an SPDConv [23] downsampling enhancement module and a CSP-OmniKernel feature fusion module, corresponding to the feature compression stage and the multiscale receptive field aggregation stage, respectively. Specifically, SPDConv is designed to replace conventional stride-2 convolutions by reducing semantic loss through four-dimensional reencoding of input features. Given an arbitrary input feature map

$$X_{\text{in}} \in \mathbb{R}^{C \times H \times W},$$

SOAPN performs even–odd index partitioning as follows:

$$X_{\text{SPD}} = \text{Conv} \left(\begin{bmatrix} X^{0,0} & \parallel & X^{1,0} & \parallel & X^{0,1} & \parallel & X^{1,1} \end{bmatrix} \right),$$

where

$$X^{i,j} = X_{\text{in}}[:, :, i :: 2, j :: 2]$$

The working principle of SPDConv is illustrated in Figure 5.

SPDConv first splits the spatial information into the channel dimension via a space-to-depth operation without losing the pixel information; then, a convolution with stride 1 is used to fuse and compress features to avoid loss of information due to traditional stride convolutions. This design helps local context modeling and is especially useful in small object detection.

A CSP-OmniKernel module is subsequently introduced in the multiscale feature fusion stage. This module leverages the cross-stage partial (CSP) mechanism [24] to split the input channels into two parts: one part is fed into a dilated convolution group for multiscale feature fusion, while the other part is preserved through identity mapping to retain the

original texture information. This design effectively prevents the loss of structural details caused by deep convolutional operations. The process is defined as follows:

$$X_c = [X_e \parallel X_r] \text{ s.t. } C_e = \lfloor C \cdot e \rfloor, C_r = C - C_e$$

$$Y = \text{Conv}_2(\mathcal{O}(X_e) \parallel X_r)$$

where X_e and X_r denote the enhanced and residual feature subsets, respectively; C represents the number of input channels; e represents the channel split ratio; and $\parallel$ denotes channelwise concatenation.

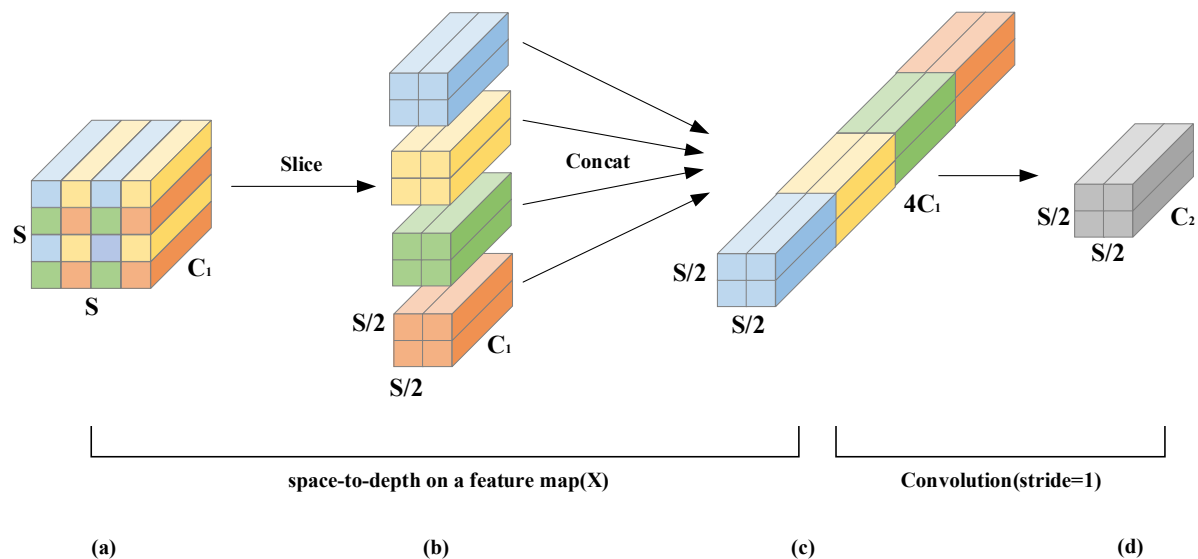


Figure 5. Schematic diagram of SPDCConv. (a) Original Spatial Information; (b) Space-to-Depth Transformed Information; (c) Channel Dimension; (d) Convolutional Feature Map.

The operator $\mathcal{O}(\cdot)$ denotes the OmniKernel multiscale processing stream [25], which can be further expressed as:

$$\mathcal{O}(X_e) = \sum_{k \in K} \sum_{d \in D} \text{Conv}_{(k,d)}(X_e),$$

where K represents the number of convolution kernels and D denotes the number of dilation rates. By combining heterogeneous kernels with dilation factors, OmniKernel is able to expand the receptive field and capture context information at multiple scales.

The working principle of OmniKernel is illustrated in Figure 6.

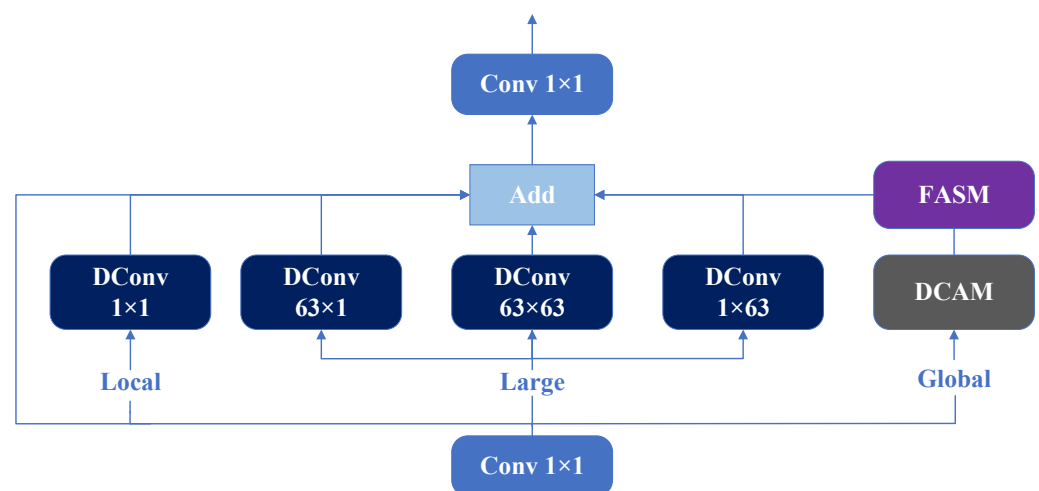


Figure 6. Schematic diagram of OmniKernel.

The OmniKernel module adopts a multibranch architecture to model receptive fields at multiple scales. Specifically, the local branch employs a 1×1 depthwise convolution to extract fine-grained features, whereas the large-scale branch captures long-range spatial dependencies via anisotropic large-kernel depthwise convolutions (63×1 , 1×63 , and 63×63). The global branch introduces a dual-domain channel attention module (DCAM) and a frequency-based spatial attention module (FSAM) to enhance the responses of the salient regions. Finally, a 1×1 convolution is applied for feature fusion, and a residual connection is adopted to stabilize training. This design enables global context modeling while maintaining computational efficiency.

The convolution kernel set $K = \{3, 5, 7\}$ and the dilation rates $D = \{1, 2, 3\}$ jointly construct a dynamic receptive field, enabling the network to simultaneously capture local structural textures and long-range contextual dependencies. Unlike conventional unidirectional pyramid propagation, SOAPN introduces multilevel, weight-sharing cross-scale skip connections within the feature fusion pathway. This design is better suited to the rapidly changing viewpoints encountered in high-speed driving scenarios and effectively enhances the consistency of semantic and texture representations throughout the network.

3.3. TADAHead

In this paper, we further improve the detection head and propose a new TADAHead to solve several limitations of YOLOv8 in autonomous driving: low-texture feature adaptation, strong coupling between classification and regression, and poor spatial feature alignment. Its overall architecture is illustrated in Figure 7. Conventional YOLO detection heads rely on shared feature representations to perform both classification and bounding box regression at the output, leading to semantic classification versus precise localization of feature representation. This problem becomes especially challenging in deep low-light scenarios (such as dense small-object distributions, heavy occlusion, or low-intensity conditions), where features in the same location must encode class semantics at the same time while maintaining sufficient sensitivity to boundary deformations, which leads to strong gradient competition and error coupling.

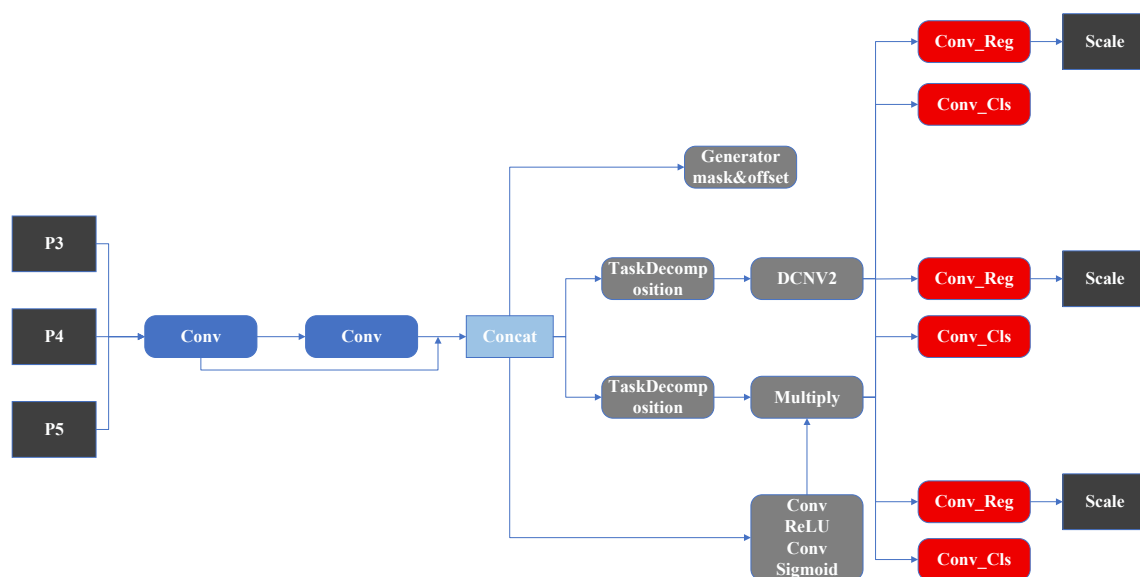


Figure 7. Structural diagram of TADAHead.

To solve these problems, TADAHead employs multilevel dynamic task decomposition, learnable scale-aware weighting and dynamic offset-aligned convolution to achieve adaptive feature modeling, task decoupling, and spatial recalibration and helping the detection head obtain more discriminative task-specific representations at prediction time.

Specifically, TADAHead first receives multiscale feature maps from the backbone and neck networks. Assuming that there are n_l input feature levels, the input can be denoted as $\mathbf{x} = \{x_1, x_2, \dots, x_{n_l}\}$. Each scale feature is initially processed by a shared convolutional stack (share_conv), which consists of a two-stage convolutional structure. Group normalization [26] is employed instead of batch normalization [27] to ensure statistical stability under small-batch training conditions. This shared encoding extracts a unified base representation for both classification and regression tasks, reducing parameter redundancy while enhancing cross-scale feature consistency.

Next, we provide a task decomposition module that divides task-independent shared features into two task-specific branches, with global average pooling as additional guides. cls_decomp focuses on improving classification-related features, and reg_decomp improves localization-related representations. With this approach, the classification gradients do not affect the regression features of typical coupled detection heads. From a theoretical point of view, the suppression effect of high-variance positive GIoU samples by classification optimization can be avoided, and thus, the localization accuracy is improved.

To address insufficient spatial alignment caused by fixed convolutional sampling locations, DyDCNv2 dynamic deformable convolution is incorporated into the regression branch. This module combines the advantages of dynamic convolution [28] and deformable convolution [29] and introduces a spatial_conv_offset subnetwork to predict pixel-level offsets and modulation masks, enabling dynamic spatial feature resampling. The offset dimension is defined as $2 \times 3 \times 3$, corresponding to adaptive adjustment of the nine sampling points in deformable convolution. This design allows regression features to focus more effectively on object boundaries, corners, and fine-grained geometric regions. Compared with the fixed-kernel structure of the original YOLOv8 head, the proposed approach significantly improves robustness under occlusion, scale variation, and shape deformation.

In addition, a CLS-Prob capability modulation mechanism is proposed to further enhance classification performance. This module employs cls_prob_conv1 and cls_prob_conv2 to predict classification confidence priors, which are activated via a sigmoid [30] function and then applied to the classification feature maps via element-wise multiplication. This attention-guided semantic modulation effectively suppresses low-confidence noisy activations and enhances class discrimination ability.

At the output stage, TADAHead generates distributional predictions of size $4 \times \text{reg_max}$ using the cv2 layer, which are decoded into continuous bounding box coordinates via distribution focal loss (DFL). The decoded predictions are then combined with dynamically generated anchors and strides to recover absolute-scale bounding boxes. During inference, predictions from all feature levels are aggregated into a unified output tensor of the form

$$Y = [x, y, w, h, p_1, p_2, \dots, p_{nc}],$$

where the first four elements represent bounding box coordinates and the remaining nc elements correspond to class probabilities. TADAHead supports both training and deployment modes and is compatible with lightweight inference frameworks such as TFLite and EdgeTPU, enabling direct application in embedded and real-time autonomous driving systems.

4. Experiments and Analysis

4.1. Dataset

The dataset used in this study is derived from the publicly available KITTI benchmark dataset [31]. KITTI was jointly established by the Karlsruhe Institute of Technology (KIT) and the Toyota Technological Institute at Chicago (TTIC) and has become one of the most influential large-scale evaluation benchmarks for computer vision algorithms in

autonomous driving scenarios. The dataset contains real-world images collected from urban, rural, and highway environments. Each image may include up to 15 vehicles and 30 pedestrians, accompanied by varying degrees of occlusion and truncation. The annotated categories cover eight classes, namely, Car, Van, Truck, Pedestrian, Person Sitting, Cyclist, Tram, and Misc. Owing to its comprehensive coverage of common autonomous driving object categories, the KITTI dataset is well suited for evaluating the detection accuracy of the proposed model.

In this work, a total of 7481 annotated images from the KITTI dataset are selected, with representative examples shown in Figure 8. This dataset covers eight target categories, and the category distribution is severely imbalanced, which may affect the model's generalization ability. Therefore, to ensure the validity of the experiment and the balance of the data, this paper filters and reclassifies the original KITTI dataset, merging Car, Truck, Van, and Tram into the "Car" class, as well as merging Person Sitting and Pedestrian into the "Pedestrian" class, while ignoring the Misc category, the final result is a target detection dataset for three main categories: Car, Pedestrian, and Cyclist. In order to ensure the diversity of training data and avoid overfitting during the data partitioning process, this paper randomly shuffles the processed dataset and divides the data into training set, validation set and test set in a ratio of 8:1:1.



Figure 8. Sample images from the dataset.

4.2. Experimental Environment and Training Settings

All the experiments are run on Linux. The computer consists of an AMD Ryzen 7 7745HX CPU, 16 GB of RAM, and an NVIDIA GeForce RTX 4090 GPU with 24 GB of video memory. The deep learning framework used is Python 3.8, PyTorch 2.2.2, and CUDA 12.1. The experimental environment is summarized in Table 1.

Table 1. Experimental environment.

Experimental Environment	Value
Processor	R7-7745HX
Operating System	Linux
Memory	16 GB
GPU	RTX 4090
GPU Memory	24 GB
Programming Language	Python 3.8
Deep Learning Framework	PyTorch 2.2.2
Deep Learning Toolkit	CUDA 12.1

This study uses YOLOv8s as the baseline model. The input image resolution is 640×640 . The SGD optimizer is trained with an initial learning rate of 0.01, momentum of 0.9, and weight decay of 0.005. The batch size is 32, and the model is trained for 250 epochs. The training hyperparameter settings are shown in Table 2.

Table 2. Training parameter settings.

Parameter	Value
Input Image Size	640 × 640
Learning Rate	0.01
Weight Decay	0.005
Momentum	0.9
Optimizer	SGD
Batch Size	32
Training Epochs	250

4.3. Evaluation Metrics

To measure the performance of our model, several popular metrics, such as precision (P), recall (R), mean average precision (mAP), $GFLOPs$, parameters, and frames per second (FPS), are also used. These metrics are defined and computed as follows.

The precision (P) is the ratio of correctly predicted positive samples to all predicted positive samples, which can be interpreted as

$$P = \frac{TP}{TP + FP}$$

where TP denotes true positives and FP denotes false positives.

Recall (R) is the ratio of correctly predicted positive samples to all real positive samples:

$$R = \frac{TP}{TP + FN}$$

where FN denotes false negatives.

The $F1$ score is the harmonic mean of precision and recall reflecting the performance of the model, and the following is the $F1$ score:

$$F_1 = \frac{1}{\frac{1}{R} + \frac{1}{P}} = \frac{2 \times P \times R}{P + R}$$

mAP (mean average precision) is the performance of all classes. AP provides detection accuracy for one class and mAP for all categories. The formulas are as follows:

$$AP = \sum_{i=1}^n (R_i - R_{i-1}) \cdot P_{i-1}$$

$$mAP = \frac{1}{C} \sum_{s=1}^C AP_s$$

where n is the number of recall levels, C is the total number of objects, and AP_s is the average precision of the s -th class.

$mAP@0.5$ refers to the mean Average Precision calculated at an IoU (Intersection over Union) threshold of 0.5. $mAP@0.5 : 0.95$ is obtained by calculating the Average Precision (AP) at multiple IoU thresholds ranging from 0.5 to 0.95 with a step of 0.05, and then averaging the results.

$GFLOPs$ (Giga Floating Point Operations) represent the computational complexity of a model during inference, defined as the total number of floating-point operations divided by 10^9 . It can be expressed as

$$GFLOPs = \frac{FLOPs}{10^9}$$

where $FLOPs$ denotes the total number of floating-point operations required for a forward pass of the network.

Parameters refer to the total number of learnable weights in a neural network, which determine the model's capacity and storage requirements. It can be expressed as

$$Parameters = \sum_l W_l$$

where W_l denotes the number of learnable parameters in layer l .

In addition, *FPS* (Frames Per Second) represents the inference speed of a model, indicating the number of images processed per second. It can be calculated as

$$FPS = \frac{N}{T}$$

where N denotes the number of processed images and T represents the total inference time in seconds.

4.4. Comparative Experiments

To objectively verify the reliability and effectiveness of the proposed SST-YOLO model in autonomous driving object detection tasks, several representative object detection algorithms, including SSD, Faster R-CNN, YOLOv3, YOLOv5, YOLOv7, YOLOv8, and YOLOv10, were selected for comparison. All compared models were trained and evaluated under the same experimental settings and training parameters to ensure fairness. The quantitative comparison results are reported in Table 3.

Table 3. Comparison Results of Different Object Detection Models.

Model	P/%	R/%	mAP@0.5/%	mAP@0.5–0.95/%
SSD	86.4	79.4	83.3	77.9
Faster RCNN	84.5	77.7	80.9	67.5
YOLOv3	89.3	83.3	86.9	64.7
YOLOv5	90.3	84.7	88.6	65.5
YOLOv7	90.5	84.1	87.4	64.5
YOLOv8	90.1	82.9	89.2	65.1
YOLOv10	91.1	85.2	88.9	65.7
SST-YOLO	93.4	84.4	91.7	69.2

As shown in Table 3, the proposed SST-YOLO achieves an mAP@0.5 of 91.7%, which is higher than those of SSD, Faster R-CNN, YOLOv3, YOLOv5, YOLOv7, YOLOv8, and YOLOv10 by 8.4%, 10.8%, 4.8%, 3.1%, 4.3%, 2.5% and 2.8%, respectively. SST-YOLO has the highest precision (93.4%) and competitive recall (84.4%), indicating higher sensitivity to small-scale and high-level targets. These results validate the performance of the proposed improvements and demonstrate that SST-YOLO outperforms other detectors in autonomous driving scenarios.

4.5. Ablation Experiments

To comprehensively evaluate the effectiveness of each proposed module, a series of ablation experiments were conducted based on the YOLOv8s baseline model. The experimental results are shown in Table 4, covering metrics such as mAP@0.5, mAP@0.5–0.95, FPS, GFLOPs, and parameter count, providing a comprehensive evaluation from the perspectives of detection accuracy, computational complexity, and real-time performance.

The baseline model YOLOv8s achieved an mAP@0.5 of 89.2%, an mAP@0.5–0.95 of 65.1%, GFLOPs of 28.4, 11.13 M parameters, and an FPS of 512.89.

After introducing the SCC module, mAP@0.5 and mAP@0.5–0.95 improved to 90.2% and 65.5%, respectively. This improvement is mainly attributed to the enhanced edge and fine-grained structure representation resulting from gradient information. Meanwhile, GFLOPs decreased by 1.6 and the number of parameters decreased by 0.64 M, while the

FPS dropped to 438.58 due to additional gradient computation. With only the SOAPN module introduced, mAP@0.5 and mAP@0.5–0.95 improved to 90.8% and 67.3%, respectively, indicating that multi-scale feature fusion enhanced the expressive power of targets at different scales. Its multi-branch structure introduced some computational overhead (GFLOPs increased to 29.9), but the number of parameters decreased slightly and the FPS remained at a high level. With only TADAHead introduced, mAP@0.5 and mAP@0.5–0.95 improved to 90.5% and 67.3%, respectively. This improvement stemmed from the decoupling of classification and localization tasks, while the optimization of the detection head structure significantly reduced the number of parameters (9.43 M) and GFLOPs (25.8), resulting in higher FPS.

Table 4. Ablation experiment results.

Model	mAP@0.5/%	mAP@0.5–0.95/%	FPS	GFLOPs	Params (M)
YOLOv8s	89.2	65.1	512.89	28.4	11.1
YOLOv8s + SCC	90.2	65.5	438.58	26.8	10.5
YOLOv8s + SOAPN	90.8	67.3	504.39	29.9	10.3
YOLOv8s + TADAHead	90.5	67.3	599.01	25.8	9.4
YOLOv8s + SCC + SOAPN	91.6	68.6	567.57	28.4	9.7
YOLOv8s + SCC + TADAHead	91.4	68.5	562.58	24.3	8.8
YOLOv8s + SOAPN + TADAHead	91.5	68.1	554.70	28.4	9.2
YOLOv8s + SCC + SOAPN + TADAHead	91.7	69.2	525.13	26.8	8.6

In the combined model, SCC + SOAPN achieved 91.6% mAP@0.5 and 68.6% mAP@0.5–0.95, indicating that edge information was more fully utilized after multi-scale fusion; SCC + TADAHead achieved 91.4% and 68.5%, showing that enhanced features were more effectively utilized in the decoupled detection head; SOAPN + TADAHead achieved 91.5% and 68.1%, demonstrating the combined effect of multi-scale information and task optimization, while keeping complexity under control.

Finally, after integrating SCC + SOAPN + TADAHead, the model achieved optimal performance (91.7% mAP@0.5 and 69.2% mAP@0.5–0.95). While improving accuracy, the number of parameters decreased to 8.57 M, GFLOPs were reduced to 26.8, and FPS remained at a high level.

4.6. Model Generalization Experiments

The SODA10M dataset [32], which was jointly released by Huawei Noah’s Ark Lab and Sun Yat-sen University, is a new-generation semi/self-supervised 2D benchmark dataset for autonomous driving. The images are collected from 32 different cities, covering most regions of China, and include a wide variety of driving scenarios, such as urban roads, highways, suburban roads, and industrial parks. In addition, the dataset spans diverse weather conditions (sunny, cloudy, rainy, and snowy) and time periods (daytime, nighttime, and dawn/dusk), providing strong diversity and complexity. Owing to these characteristics, SODA10M is well suited for evaluating the generalization capability of object detection models.

A total of 10,000 annotated images from the SODA10M dataset are selected to fit the training, validation and test sets at an 8:1:1 ratio. The trained models are compared on the test set for generalization. To investigate the external validity of the proposed SST-YOLO, comparisons with several popular object detectors are conducted, and the quantitative results are shown in Table 5.

Table 5. Model generalization experiment results.

Model	P/%	R/%	mAP@0.5/%	mAP@0.5–0.95/%
YOLOv5	79.2	71.3	83.1	55.1
YOLOv7	81.1	73.7	82.1	54.6
YOLOv8	80.9	73.8	82.3	56.4
SST-YOLO	81.4	73.3	87.8	60.7

As shown in Table 5, SST-YOLO achieves the best accuracy, with mAP@0.5 and mAP@0.5–0.95 of 87.8% and 60.7%, respectively. While SST-YOLO provides a lower recall than YOLOv8 does, its precision and mAP are significantly higher than those of YOLOv8. Compared with YOLOv7, SST-YOLO results in lower recall and higher scores for other evaluation metrics. Compared with YOLOv5, SST-YOLO yields better performance in terms of precision, recall, mAP@0.5, and mAP@0.5–0.95. These results show that SST-YOLO offers competitive and stable performance across all driving environments and proves its generalizability and external validity.

4.7. Visualization Analysis

To further verify the reliability of SST-YOLO in autonomous driving scenarios, qualitative visualization analysis is conducted, as illustrated in Figure 9. In the figure, the first column shows the detection results of the YOLOv8 baseline model, whereas the second column presents the results obtained by the proposed SST-YOLO.



Figure 9. Visualization analysis of the experimental results. (a) Images detected by YOLOv8. (b) Images detected by SST-YOLO.

In the first row, YOLOv8 failed to detect small-sized cars and pedestrians in the distance, exhibiting a significant false negative problem. This is mainly due to the small size and weak texture information of distant targets, which are easily lost during feature extraction and downsampling. In contrast, SST-YOLO successfully detected these targets. This is primarily due to the Sobel branch introduced by the SCC module, which enhances

edge and high-frequency structural information, making the model more sensitive to targets with weak texture. Simultaneously, the SOAPN module effectively preserves the fine-grained information of small targets through multi-scale feature enhancement and cross-layer fusion, thus significantly reducing the false negative rate.

In the second row, YOLOv8's detection of the car in the lower right corner of the image is incomplete, with local false negatives appearing in the front of the car. This problem typically stems from the target being located at the image boundary, resulting in incomplete feature representation. SST-YOLO can detect the target more completely, primarily due to SOAPN's cross-scale information compensation mechanism, which enhances the feature representation of the boundary region. Simultaneously, the dynamic alignment and regression optimization mechanisms in TADAHead improve the localization ability of the bounding boxes, making the detection results more complete and accurate.

In the third row, YOLOv8 misdetects the leftmost trash can in the image as a car, a typical false positive problem. This reflects the model's insufficient ability to discriminate category semantics in complex scenes. In contrast, SST-YOLO effectively avoids this false positive, mainly thanks to TADAHead's task decoupling mechanism, which makes the classification and regression processes more independent, thus reducing category confusion. Furthermore, the structural edge information provided by SCC also helps the model more accurately distinguish between real vehicles and similar-looking objects.

In the fourth row, YOLOv8 generates redundant detection boxes in the complex background region in the center of the image, incorrectly detecting two pedestrians as three targets, exhibiting a coexistence of duplicate detections and false positives. This is typically related to unstable feature representation and insufficient target discrimination ability. In contrast, SST-YOLO can effectively suppress redundant bounding boxes and retain only the true targets. This improvement mainly stems from TADAHead's dynamic spatial alignment and feature recalibration capabilities, which enhances the consistency of target localization; at the same time, the multi-scale contextual information provided by SOAPN enhances the ability to distinguish dense targets, thereby reducing duplicate detections.

These visualization results show that SST-YOLO demonstrates significant advantages in target detection tasks: SCC strengthens edge and detail perception capabilities, SOAPN improves multi-scale feature representation and small target detection capabilities, and TADAHead optimizes the collaborative mechanism between classification and localization. The synergistic effect of these three technologies enables the model to effectively reduce both false negatives and false positives in complex autonomous driving environments, significantly improving detection accuracy and robustness.

5. Conclusions

This paper proposes an object detection model for autonomous driving called SST-YOLO. On the basis of YOLOv8, we implement SCC in the backbone, replace the neck with SOAPN, and deploy the TADAHead detector head.

The SCC module concatenates the features acquired with SobelConv and standard convolution, allowing the learned representations to capture all edge information and spatial semantic features to accurately describe the image content. SOAPN generalizes bidirectional feature representations by employing heterogeneous-scale convolutions, multidimensional receptive field fusion and lightweight cross-layer aggregation, thereby increasing global semantic information and preserving local detail. As a result, the detection network is more sensitive to small targets, has better long-distance discrimination ability and is more robust to occlusion in autonomous driving. TADAHead improves detection performance by using multilevel dynamic decomposition, learnable scale weights and dynamic offset

aligned convolutions with feature adaptivity, task decoupling and spatial recalibration, which effectively improves the discriminative performance of the detection head.

Experiments show that in the autonomous driving object detection task, SST-YOLO achieves notable improvements over the YOLOv8 baseline, with the proposed method increasing mAP@0.5 by 2.5% and mAP@0.5:0.95 by 4.1%. Meanwhile, SST-YOLO maintains competitive computational efficiency, with its FPS slightly increasing (from 512.89 to 525.13), while GFLOPs decreases from 28.4 to 26.8, and the number of parameters is reduced from 11.1 M to 8.6 M. These performance improvements are sufficient to demonstrate that the model can meet the requirements of autonomous driving perception well.

The method proposed in this study has only been validated on publicly available datasets, which has certain limitations. Therefore, in future work, we will further validate it experimentally in a real-world environment. In addition, we will also improve the robustness of autonomous driving object detection algorithms under challenging conditions (microsensor noise, strong light, and backlighting), as well as model lightweighting and in-vehicle deployment optimization, which will lead to more widespread application of autonomous vehicle object detection.

Author Contributions: N.Z. and Q.D. are responsible for the design and execution of the experiments, as well as the writing of the manuscript. W.B., R.Z. and Y.L. are responsible for preparing the figures and tables. C.S. and S.Z. are responsible for organizing the experimental data. J.Z. is responsible for providing experimental equipment and technical support. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The datasets used and analyzed during the current study are publicly available and can be accessed from <http://www.semantic-kitti.org/dataset.html#download> (accessed on 15 December 2025).

Acknowledgments: We would like to express our deepest gratitude to all those who have contributed to the completion of this research and the writing of this paper.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Yurtsever, E.; Lambert, J.; Carballo, A.; Takeda, K. A survey of autonomous driving: Common practices and emerging technologies. *IEEE Access* **2020**, *8*, 58443–58469. [\[CrossRef\]](#)
2. Voulodimos, A.; Doulamis, N.; Doulamis, A.; Protopapadakis, E. Deep learning for computer vision: A brief review. *Comput. Intell. Neurosci.* **2018**, *2018*, 7068349. [\[CrossRef\]](#)
3. Zou, Z.; Chen, K.; Shi, Z.; Guo, Y.; Ye, J. Object detection in 20 years: A survey. *Proc. IEEE* **2023**, *111*, 257–276. [\[CrossRef\]](#)
4. Guo, Y.; Wang, H.; Hu, Q.; Liu, H.; Liu, L.; Bennamoun, M. Deep learning for 3d point clouds: A survey. *IEEE Trans. Pattern Anal. Mach. Intell.* **2020**, *43*, 4338–4364. [\[CrossRef\]](#) [\[PubMed\]](#)
5. Liu, Y.; Sun, P.; Wergeles, N.; Shang, Y. A survey and performance evaluation of deep learning methods for small object detection. *Expert Syst. Appl.* **2021**, *172*, 114602. [\[CrossRef\]](#)
6. Yang, X.; Wu, W.; Liu, K.; Kim, P.W.; Sangaiah, A.K.; Jeon, G. Long-distance object recognition with image super resolution: A comparative study. *IEEE Access* **2018**, *6*, 13429–13438. [\[CrossRef\]](#)
7. LeCun, Y.; Bengio, Y.; Hinton, G. Deep learning. *Nature* **2015**, *521*, 436–444. [\[CrossRef\]](#) [\[PubMed\]](#)
8. Alzubaidi, L.; Zhang, J.; Humaidi, A.J.; Al-Dujaili, A.; Duan, Y.; Al-Shamma, O.; Santamaria, J.; Fadhel, M.A.; Al-Amidie, M.; Farhan, L. Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions. *J. Big Data* **2021**, *8*, 53. [\[CrossRef\]](#) [\[PubMed\]](#)
9. Khan, F.A.; Gumaie, A.; Derhab, A.; Hussain, A. A novel two-stage deep learning model for efficient network intrusion detection. *IEEE Access* **2019**, *7*, 30373–30385. [\[CrossRef\]](#)

10. Wang, T.; Yang, F.; Tsui, K.L. Real-time detection of railway track component via one-stage deep learning networks. *Sensors* **2020**, *20*, 4325. [[CrossRef](#)] [[PubMed](#)]
11. Chen, L.; Wu, P.; Chitta, K.; Jaeger, B.; Geiger, A.; Li, H. End-to-end autonomous driving: Challenges and frontiers. *IEEE Trans. Pattern Anal. Mach. Intell.* **2024**, *46*, 10164–10183. [[CrossRef](#)] [[PubMed](#)]
12. Kanopoulos, N.; Vasanthavada, N.; Baker, R.L. Design of an image edge detection filter using the Sobel operator. *IEEE J. Solid-State Circuits* **1988**, *23*, 358–367. [[CrossRef](#)]
13. Bappy, J.H.; Roy-Chowdhury, A.K. CNN based region proposals for efficient object detection. In *2016 IEEE International Conference on Image Processing (ICIP)*; IEEE: New York, NY, USA, 2016; pp. 3658–3662.
14. Uijlings, J.R.; Van De Sande, K.E.; Gevers, T.; Smeulders, A.W. Selective search for object recognition. *Int. J. Comput. Vis.* **2013**, *104*, 154–171. [[CrossRef](#)]
15. Neubeck, A.; Van Gool, L. Efficient non-maximum suppression. In *18th International Conference on Pattern Recognition (ICPR'06)*; IEEE: New York, NY, USA, 2006; Volume 3, pp. 850–855.
16. Girshick, R. Fast r-cnn. In *Proceedings of the IEEE International Conference on Computer Vision*; IEEE: New York, NY, USA, 2015; pp. 1440–1448.
17. He, K.; Zhang, X.; Ren, S.; Sun, J. Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE Trans. Pattern Anal. Mach. Intell.* **2015**, *37*, 1904–1916. [[CrossRef](#)] [[PubMed](#)]
18. Chauhan, V.K.; Dahiya, K.; Sharma, A. Problem formulations and solvers in linear SVM: A review. *Artif. Intell. Rev.* **2019**, *52*, 803–855. [[CrossRef](#)]
19. Ren, S.; He, K.; Girshick, R.; Sun, J. Faster R-CNN: Towards real-time object detection with region proposal networks. *IEEE Trans. Pattern Anal. Mach. Intell.* **2016**, *39*, 1137–1149. [[CrossRef](#)] [[PubMed](#)]
20. Redmon, J.; Divvala, S.; Girshick, R.; Farhadi, A. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*; IEEE: New York, NY, USA, 2016; pp. 779–788.
21. Liu, W.; Anguelov, D.; Erhan, D.; Szegedy, C.; Reed, S.; Fu, C.Y.; Berg, A.C. SSD: Single shot multibox detector. In *European Conference on Computer Vision*; Springer International Publishing: Cham, Switzerland, 2016; pp. 21–37.
22. Liu, S.; Qi, L.; Qin, H.; Shi, J.; Jia, J. Path aggregation network for instance segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*; IEEE: New York, NY, USA, 2018; pp. 8759–8768.
23. Sunkara, R.; Luo, T. No more strided convolutions or pooling: A new CNN building block for low-resolution images and small objects. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*; Springer Nature: Cham, Switzerland, 2022; pp. 443–459.
24. Wang, C.Y.; Liao, H.Y.M.; Wu, Y.H.; Chen, P.Y.; Hsieh, J.W.; Yeh, I.H. CSPNet: A new backbone that can enhance learning capability of CNN. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*; IEEE: New York, NY, USA, 2020; pp. 390–391.
25. Cui, Y.; Ren, W.; Knoll, A. Omni-kernel network for image restoration. In *Proceedings of the AAAI Conference on Artificial Intelligence*; IEEE: New York, NY, USA, 2024; Volume 38, pp. 1426–1434.
26. Wu, Y.; He, K. Group normalization. In *Proceedings of the European Conference on Computer Vision (ECCV)*; Springer: Cham, Switzerland, 2018; pp. 3–19.
27. Bjorck, N.; Gomes, C.P.; Selman, B.; Weinberger, K.Q. Understanding batch normalization. *Adv. Neural Inf. Process. Syst.* **2018**, *31*, 1–12.
28. Chen, Y.; Dai, X.; Liu, M.; Chen, D.; Yuan, L.; Liu, Z. Dynamic convolution: Attention over convolution kernels. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*; IEEE: New York, NY, USA, 2020; pp. 11030–11039.
29. Zhu, X.; Hu, H.; Lin, S.; Dai, J. Deformable convnets v2: More deformable, better results. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*; IEEE: New York, NY, USA, 2019; pp. 9308–9316.
30. Kyurkchiev, N.; Markov, S. *Sigmoid Functions: Some Approximation and Modelling Aspects*; LAP LAMBERT Academic Publishing: Saarbrücken, Germany, 2015; Volume 4, p. 34.
31. Behley, J.; Garbade, M.; Milioto, A.; Quenzel, J.; Behnke, S.; Stachniss, C.; Gall, J. Semantickitti: A dataset for semantic scene understanding of lidar sequences. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*; IEEE: New York, NY, USA, 2019; pp. 9297–9307.
32. Han, J.; Liang, X.; Xu, H.; Chen, K.; Hong, L.; Mao, J.; Ye, C.; Zhang, W.; Li, Z.; Liang, X.; et al. SODA10M: A large-scale 2D self/semi-supervised object detection dataset for autonomous driving. *arXiv* **2021**, arXiv:2106.11118.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.